

Title	AXI DMA Controller Design Guide
Project	None
Description	AXI DMA Controller Design Guide
Author	holelee
Date	19/Jul/2005

(*) AXI DMA Controller

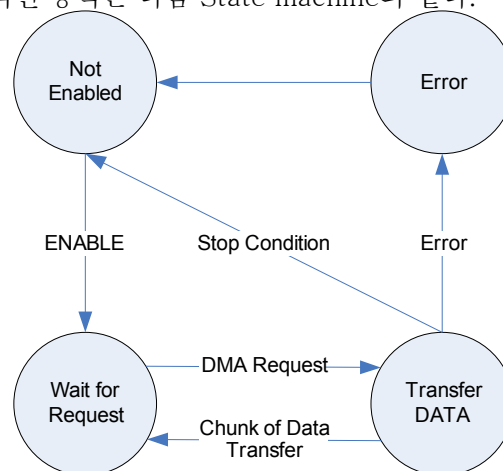
설계를 끝낸 AXI DMA Controller는 이름에서 알 수 있듯, AXI Interface를 가지는 DMA Controller이다.

(*) Feature

- Source BUS Width : BYTE/HWORD/WORD
- Destination BUS Width : BYTE/HWORD/WORD
- byte단위의 transfer count
- Automatic Pack/Unpack
- Unaligned address 지원
 - Destination address가 unaligned인 경우에는 AXI의 WSTRB를 지원하는 Destination 이어야 함
- Descriptor를 이용한 Linked list scatter/gather DMA 지원
- Little endian만 지원

(*) 기본적인 동작

DMA Controller의 기본적인 동작은 다음 State machine과 같다.



(a) 초기 상태에서는 DMA가 enable되지 않는다.

(b) Status Register를 Enable bit를 high로 하고, pending interrupt bit를 clear하면 DMA Controller가 enable되고 DMA request를 기다리게 된다. .

(c) DMA request가 들어오면 Control register의 size로 지정된 data의 갯수만큼 전송을 한 후에 Stop Condition이 아니라면 DMA request를 기다리는 상태로 돌아가게 된다. Stop Condition은 Control Register의 TransferLength가 0이 되고 Descriptor Register의 DescriptorEnd bit가 1인 경우 인데, 이 상황이면 data transfer가 끝나면 Not Enabled 상태로 천이하게 된다. Data를 transfer하는 도중에 Bus Error 상황이 있거나 아니면 register setting에서 오류가 존재하면 Error State로 천이한다. Data transfer가 성공적이라면 Source Address register, Destination Address register, Control register의 Transferlength를 update한다.

(d) Error State에서는 단순히 Error Interrupt를 발생시키고 다시 Not Enabled 상태로 돌아가게 된다.

- Descriptor Loading

Transfer Data state에서 Control register의 TransferLength가 0이고 DescriptorEnd가 0이면, Descriptor Register에 있는 Address에서 4개의 WORD를 읽어서 Source Address Register, Destination Address Register, Control Register, Descriptor Register를 write한 후에 transfer를 시작하게 된다. 단 새로 읽은 Control Register의 TransferLength가 0이면 Error State로 천이한다.

Descriptor Address에는 다음의 순서로 4 WORD가 저장되어야 한다.

첫번째 WORD : Source Address Register Contents

두번째 WORD : Destination Address Register Contents

세번째 WORD : Control Register Contents

네번째 WORD : Descriptor Register Contents

- Start Interrupt/End Interrupt/Stop Interrupt/Error Interrupt

Descriptor가 성공적으로 loading된 후에 Start Interrupt가 발생한다. Start Interrupt가 발생하여도 DMA Controller의 동작은 계속된다.

Data Transfer가 성공적으로 이루어지고 TransferLength가 0이 되면 그 때 End Interrupt가 발생한다. End Interrupt가 발생하여도 DMA Controller의 동작은 계속된다.

Stop Interrupt는 Stop condition, 즉 TransferLength가 0이 되고 DescriptorEnd가 1일 때는 Stop Interrupt가 발생하고 DMA Controller는 “Not Enabled”상태로 바뀌고 DMA Controller의 동작이 멈추게 된다. DMA Controller가 모든 data를 전송했는지 detect하기 위해서 사용하는 interrupt이다.

Error Interrupt는 DATA transfer(Descriptor Load 포함)중에 Bus error를 detect하거나 Descriptor나 register setting에 오류가 있을 때 발생한다. DMA Request가 왔는데 TransferLength가 0이고 DescriptorEnd가 1인 상황, Descriptor를 Load했는데 TransferLength가 0인 경우가 그에 해당한다.

- Trailing bytes

DMA Request가 detect되었을 때, Control register의 transfer length가 size보다 작은 경우에는 transfer length만큼의 byte만 전송이 이루어진다.

- Memory-to-Memory Transfer

메모리에서 메모리로 DMA를 하는 경우에는 별도의 DMA Request 신호가 없다. Control register의 M2M bit가 1로 해 주게 되면, 자동의 DMA Request 신호가 생성되어 전송이 이루어지게 된다.

(*) Registers

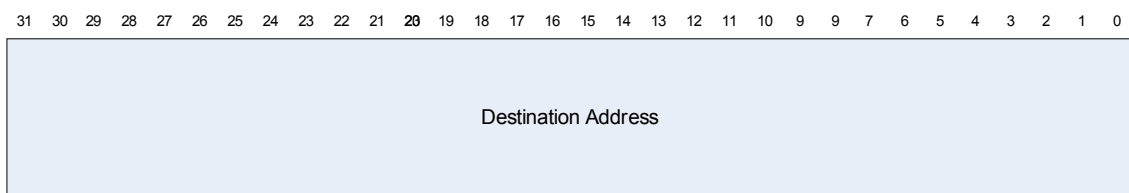
- Source Address Register(Address : PADDR[4:2] = 3'b000)



Source Address register의 각 field의 내용은 다음과 같다.

<i>bits</i>	<i>Name</i>	<i>R/W</i>	<i>Description</i>
31:0	Source Address	R/W	32 bit source address

- Destination Address Register(Address : PADDR[4:2] = 3'b001)



Destination Address register의 각 field의 내용은 다음과 같다.

<i>bits</i>	<i>Name</i>	<i>R/W</i>	<i>Description</i>
31:0	Destination Address	R/W	32 bit destination address

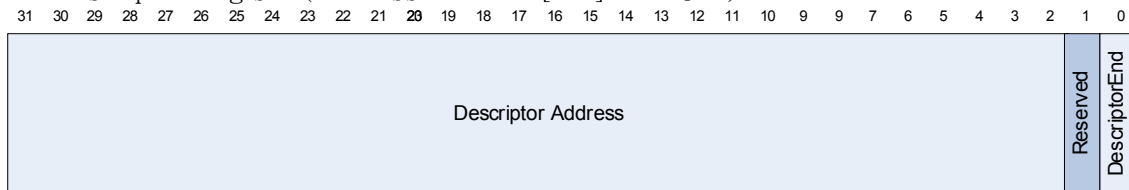
- Control Register(Address : PADDR[4:2] = 3'b010)



Control register의 각 field의 내용은 다음과 같다.

<i>bits</i>	<i>Name</i>	<i>R/W</i>	<i>Description</i>
31	StartIntEn	R/W	StartInterrupt Enable Flag 0 : StartInterrupt를 masking함 1 : StartInterrupt를 발생시킴
30	EndIntEn	R/W	EndInterrupt Enable Flag 0 : EndInterrupt를 masking함 1 : EndInterrupt를 발생시킴
29	M2M	R/W	Memory-to-Memory Transfer Flag 0 : DMA Request 신호로 DMA Controller 동작 1 : DMA Request 신호 없이 DMA Controller 동작 Memory에서 Memory로 데이터를 전송하는 DMA의 경우 외부의 DMA Request 신호와 관계 없이 전송이 계속된다. M2M bit가 1인 경우 외부 DMA Request 신호를 무시하고 계속 DMA Request가 항상 high인 것과 동일하게 동작하게 된다.
26	SrcIncr	R/W	Source Address Increment Flag 0 : 매 Transfer마다 Source Address가 증가하지 않음 1 : 매 Transfer마다 Source Address가 증가함
25:24	SrcWidth	R/W	Source Device의 BUS Width 00 : BYTE 01 : HWORD 10 : WORD 11 : reserved
22	DstIncr	R/W	Destination Address Increment Flag 0 : 매 Transfer마다 Destination Address가 증가하지 않음 1 : 매 Transfer마다 Destination Address가 증가함
21:20	DstWidth	R/W	Destination Device의 BUS Width 00 : BYTE 01 : HWORD 10 : WORD 11 : reserved
18:16	Size	R/W	Transfer Size 매 DMA Request마다 전송할 Byte 수 000 : 1 byte 001 : 2 byte 010 : 4 byte 011 : 8 byte 100 : 16 byte 101 : 32 byte other : reserved
15:0	TransferLength	R/W	전체 전송할 byte 수/현재 남아 있는 전송할 Byte 수 16bit이므로 한번에 65535Byte(64KB-1byte)를 전송할 수 있다.

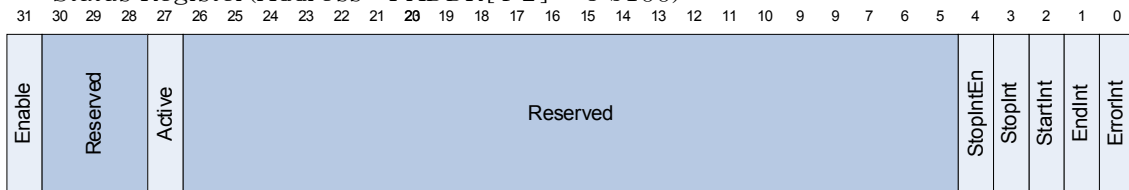
- Descriptor Register(Address : PADDR[4:2] = 3'b011)



Descriptor register의 각 field의 내용은 다음과 같다.

bits	Name	R/W	Description
31:2	Descriptor Address	R/W	다음에 Load할 Descriptor Address의 상위 30bit Descriptor Address는 32bit에 align되어야 함
0	DescriptorEnd	R/W	Descriptor End Flag 0 : Descriptor Address가 VALID함 1 : Descriptor Address가 VALID하지 않음. 즉, 현재 처리중인 Descriptor가 마지막 임

- Status Register(Address : PADDR[4:2] = 3'b100)



Status register의 각 field의 내용은 다음과 같다.

bits	Name	R/W	Description
31	Enable	R/W	Enable Flag 0 : DMA Controller Disabled 1 : DMA Controller Enabled
27	Active	R	Active Flag 0 : 현재 Transfer를 진행중이지 않음 1 : 현재 Transfer를 진행중임
4	StopIntEn	R/W	Stop Interrupt Enable Flag 0 : Stop Interrupt 발생을 disable함 1 : Stop Interrupt 발생을 enable함
3	StopInt	R/W	Stop Interrupt Status 0 : Stop Interrupt가 발생하지 않았음 1 : Stop Interrupt가 발생하였음 StopIntEn과 상관 없이 Stop Interrupt 상황에서는 1로 변경된다. 단, StopIntEn이 0이면 이 bit의 값과 상관 없이 interrupt가 발생하지 않는다. StopInt bit가 1이면 DMA Controller의 동작이 멈춘다. 따라서 DMA Controller의 동작을 계속하고 싶으면 이 bit를 0로 reset해야 한다. 1을 write하면 clear되고 0을 write하는 것은 영향없음.

<i>bits</i>	<i>Name</i>	<i>R/W</i>	<i>Description</i>
2	StartInt	R/W	Start Interrupt Status 0 : Start Interrupt가 발생하지 않았음 1 : Start Interrupt가 발생하였음 Control register의 StartIntEn이 0이면 이 bit는 set되지 않는다. 1을 write하면 clear되고 0을 write하는 것은 영향없음
1	EndInt	R/W	End Interrupt Status 0 : End Interrupt가 발생하지 않았음 1 : End Interrupt가 발생하였음 Control register의 EndIntEn이 0이면 이 bit는 set되지 않는다. 1을 write하면 clear되고 0을 write하는 것은 영향없음
0	ErrorInt	R/W	Error Interrupt Status 0 : Error Interrupt가 발생하지 않았음 1 : Error Interrupt가 발생하였음 ErrorInt bit가 1이면 DMA Controller의 동작이 멈춘다. 따라서 DMA Controller의 동작을 계속하고 싶으면 이 bit를 0로 reset해야 한다. 1을 write하면 clear되고 0을 write하는 것은 영향없음

(*) Software Model

- DMA Controller를 Enable 시키는 방법

DMA Controller를 Enable 시키기 위해서는 Status Register의 Enable bit를 high, StopInt/ErrorInt bit를 low로 만들어 주어야 한다. 따라서 Status Register의 31, 3, 0번 bit를 1로 write하면 된다.

- DMA Controller를 Disable 시키는 방법

DMA Controller는 위에서 언급한 대로 Error상황과 Stop condition에서 자동으로 disable된다. Software로 동작중인 DMA Controller를 강제로 disable시키기 위해서는 다음과 같은 절차를 따라야 한다.

(a) Status register의 Enable bit를 low로 write한다.

(b) Status register의 Active bit를 polling하여 0이 될 때까지 기다린다.

- Register access

Status Register enable bit를 제외하고 register를 write하는 것은 DMA가 disable된 상태에서만 의미가 있다. DMA Controller가 동작중에 있을 때 Register에 write를 하는 경우에는 DMA Controller가 오동작할 수 있으므로 시도하지 말아야 한다.

- Descriptor를 사용하지 않는 경우의 DMA Controller programming 방법

Descriptor를 사용하지 않는 경우 다음과 같이 programming을 할 수 있다.

(a) Source Address/Destination Address를 write한다.

(b) Control register를 programming하는데 TransferLength를 반드시 0보다 큰 값을 가지도록 해야 한다.

(c) Descriptor Register의 DescirptorEnd bit을 1로 write해 준다.

(d) Status register를 write하여 DMA Controller를 enable 시켜 준다.

- Descriptor를 사용하는 경우의 programming 방법

(a) 메모리의 특정 위치에 Descriptor를 Linked list 구조로 4 WORD씩 write한다.

(b) Control register의 TransferLength를 0으로 write한다. Memory에서 Memory로 전송하는 DMA의 경우 Control register의 M2M bit를 1로 setting해 준다.

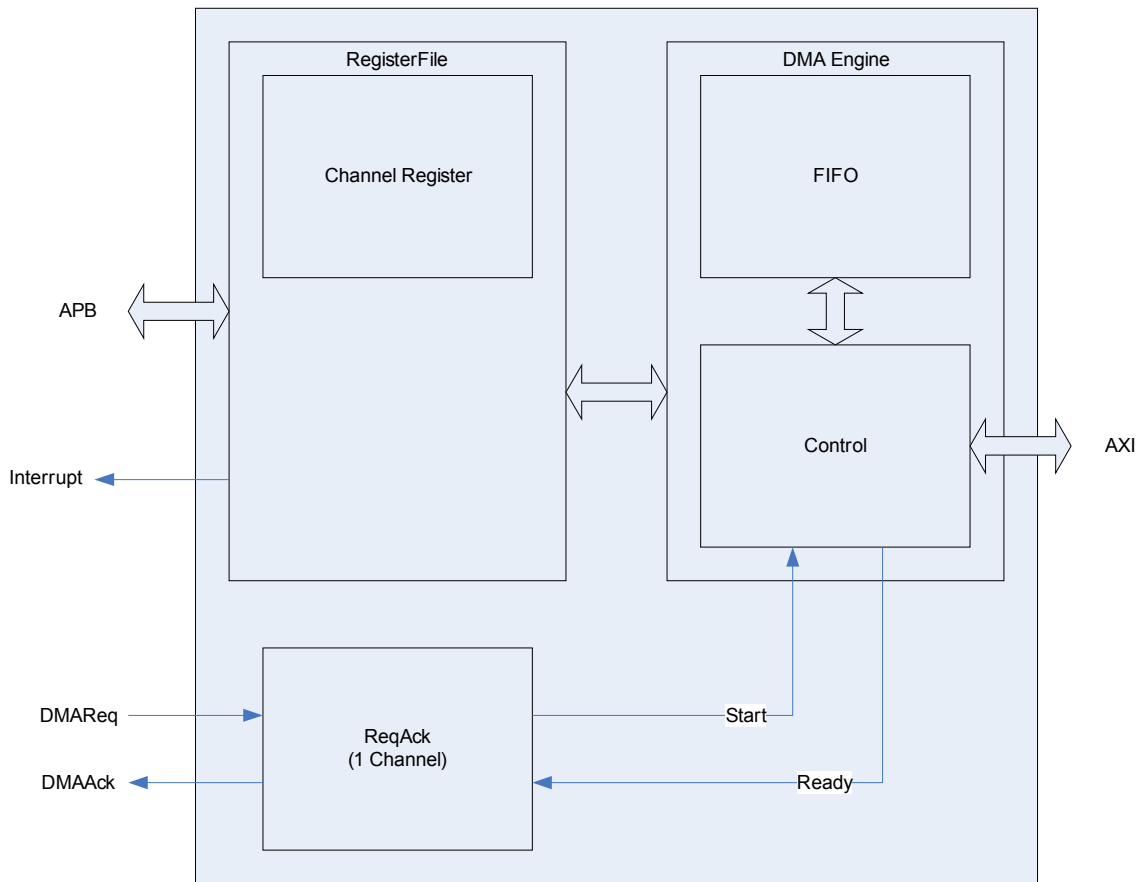
(c) Descriptor Register에 첫번째 Descriptor가 write된 address와 DescriptorEnd=0을 write한다.

(d) Status register를 write하여 DMA Controller를 enable 시켜 준다.

당연히 첫번째 Descriptor의 내용을 Memory에 write하지 않고 직접 Register에 write하여도 무방하다.

(*) 1 Channel DMA Controller의 Hardware 구조

Hardware 구조는 다음과 같다.



기본적으로 DMA Request와 DMA Acknowledge를 처리하는 block이 존재하고 RegisterFile이 있어서 APB interface를 통한 register read/write를 처리한다. 실제 DMA 처리는 DMA Engine내부에 있는 Fifo와 Control block이 처리하게 된다. 내부의 Control 신호는 Control Block이 Ready 신호를 보내게 되고, ReqAck block에서 Ready가 high일때 Start신호를 보내 동작을 시키게 된다. Channel Register와 Control block 사이에는 Register lookup과 write를 위한 interface가 존재한다.

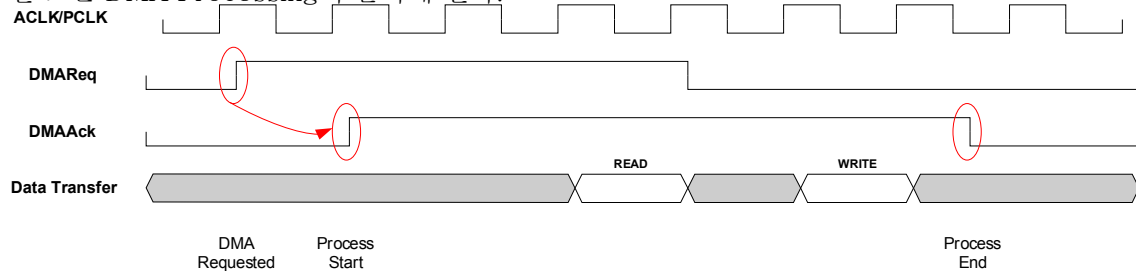
DMA Controller를 여러 peripheral이 공유하는 경우에는 DMA Req/Ack를 위한 MUX가 필요한데 그것은 포함하고 있지 않다. 이것을 지원하기 위해서는 DMA Controller 외부에 별도의 logic을 넣거나 ReqAck를 변경하여야 한다. MUX selection을 register는 Register File에 추가해야 할 필요가 있다.

- DMA Request/Acknowledge timing

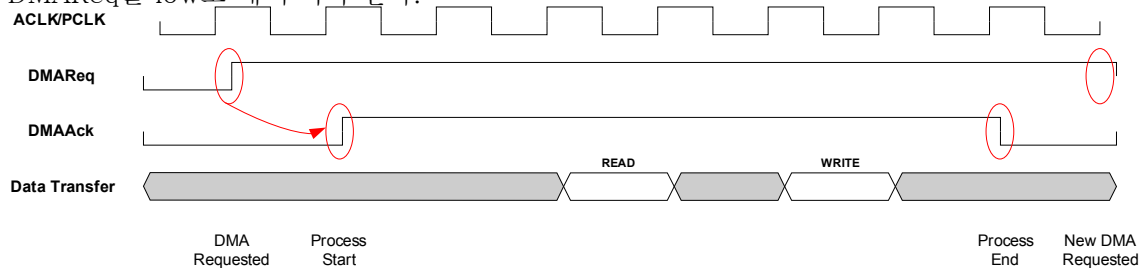
Chip 내부의 Device는 DMA Request를 자신의 FIFO level에 따라서 level sensitive하게 출력해 주면 된다. DMA Controller가 read/write를 수행함으로써 FIFO의 level이 변할 것이므로 그에 따라서 Request 신호가 자동으로 low로 바뀔 것이다. 따라서 DMA Acknowledge signal을 사용할 필요는 없다.

DMAReq 신호가 high가 되면 수 clock이 지난 후에 DMA Controller가 idle하게 되면 DMA

Ack가 high가 되어 DMA processing이 start되었음을 알려 주게 된다. DMA Ack가 low가 되면 모든 DMA Processing이 끝나게 된다.



만약 DMA Ack가 low가 될 때까지 DMAReq가 high로 유지된다면 다음 DMA Processing이 Start된다. 따라서 DMA Ack가 high가 된다음에 다음 DMA Processing을 원하지 않으면 DMAReq를 low로 해 주어야 한다.



위에서 언급한 대로 peripheral이 FIFO의 level을 이용하여 DMAReq를 생성해 주면 READ/WRITE를 수행하는 와중에 level의 변화에 따라서 자동으로 DMAReq가 low로 바뀌므로 크게 신경쓰지 않아도 된다.

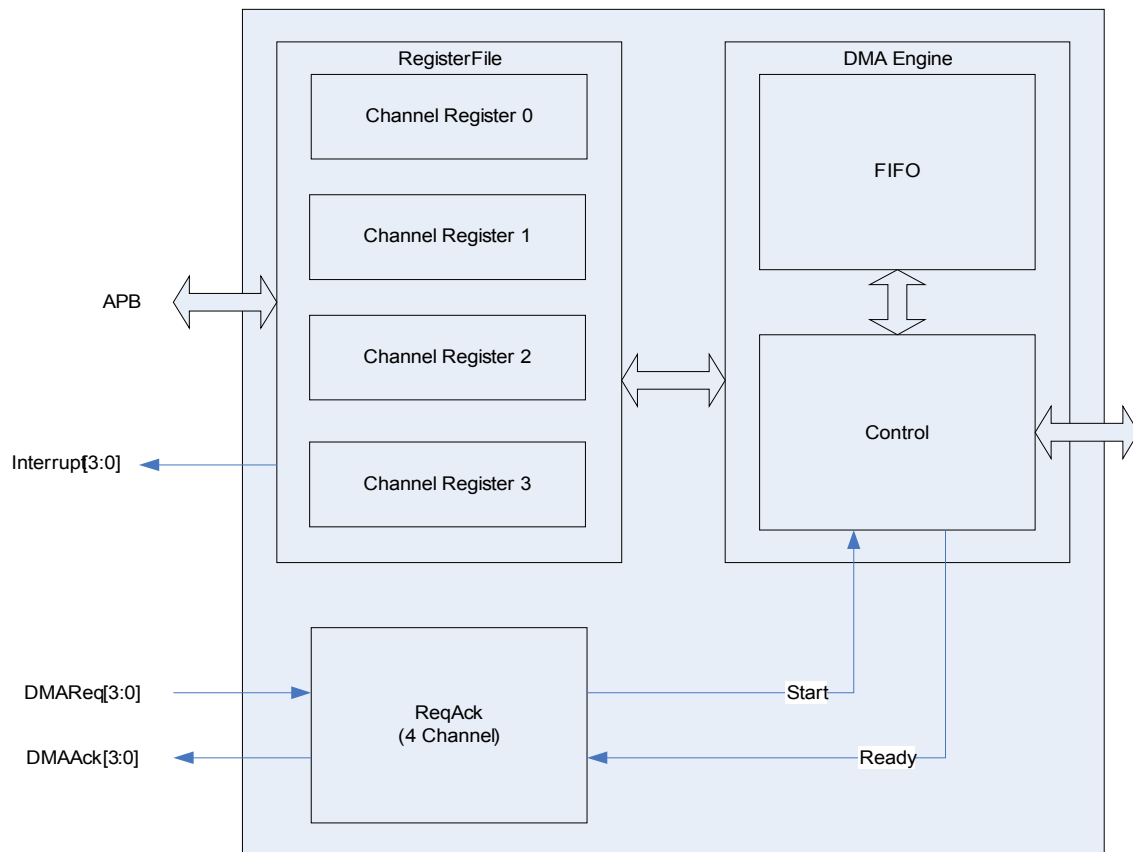
- Interrupt 출력

Interrupt 출력은 Register File에 있는 register bit를 단순히 or하여 출력하는 것이므로 active high level sensitive signal이다. 이 신호를 Interrupt Controller에 연결해 주면 된다.

(*) 4 Channel DMA Controller의 Hardware 구조

4 Channel DMA Controller의 구조는 1 Channel과 크게 다르지 않다. 단순히 Channel별로 register를 따로 가져야 하므로 4 set의 channel register가 존재하고 request/acknowledge를 처리하는 block이 4 channel에서 들어오는 DMA request를 arbitration하도록 되어 있다.(현재는 DMA Controller의 test를 위해서 round robin arbitration을 하도록 되어 있는데 이것은 추후에 priority based arbitration이나 다른 arbitration 방법으로 변경을 쉽게 할 수 있다.)

입출력 신호는 4 Channel에 맞게 Interrupt/DMAReq/DMAAck가 4 bit로 되어 있다.



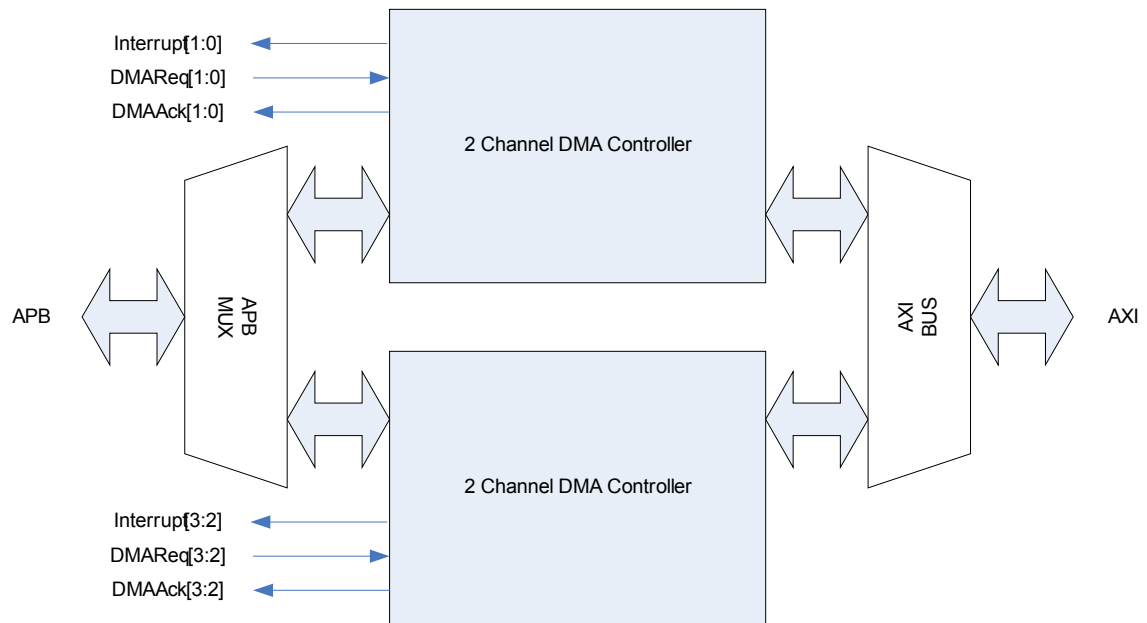
Channel Register의 접근은 PADDR[6:5]를 channel의 selection signal로 이용한다. 즉 PADDR[6:5]가 2'b00이면 0번 channel, 2'b01이면 1번 channel, 2'b10이면 2번 채널, 2'b11이면 3번 채널이다. PADDR[4:2]의 사용은 1 Channel DMA Controller와 동일하다.

(*) 2 Channel DMA Controller의 Hardware 구조

2 Channel DMA Controller의 하드웨어 구조는 4 Channel DMA Controller의 하드웨어 구조와 거의 동일하므로, 설명을 생략한다.

(*) 2x2 Channel DMA Controller의 Hardware 구조

위에서 언급한 DMA Controller는 DMA Engine이 하나 밖에 없으므로 data를 read하여 write 하는 것이 순차적으로 일어나게 된다. 이러한 순차적인 방법은 AXI의 장점인 out-of-order completion을 이용할 수 없어 효율이 떨어진다. 따라서 좀더 효율적인 DMA Controller는 다음과 같이 AXI BUS를 이용하여 구성하였다.



단, 이 방법은 Hardware size가 크다는 단점이 있다. 4 Channel DMA Controller는 DMA Engine을 하나만 가지지만 2x2 Channel의 경우 Engine을 두개 가지고 있기 때문이다. 설계된 AXI BUS는 일반적인 AXI BUS가 아니고 2x1만 처리하고, AXI Interface signal 사이에 combinational path가 존재하는 구조를 가진다. Combinational path를 가지므로 Register Slice와 연결하여 timing을 분리하여야 할 것으로 보인다.